

Day 9 — SQL

Rohit Raj Singh

Structured Query Language (SQL) is the standard language used to interact with relational databases.

- Allows users to store, retrieve, update and manage data efficiently through simple commands.
- Known for its user-friendly syntax and powerful capabilities, SQL is widely used across industries.

Type Full Form Purpose Common Commands

DDL Data Definition Language Defines database structure CREATE, ALTER, DROP, TRUNCATE

DML Data Manipulation Language Adds/changes/removes data INSERT, UPDATE, DELETE

DQL Data Query Language Retrieves data SELECT

DCL Data Control Language Controls permissions GRANT, REVOKE

TCL Transaction Control Language Controls transactions COMMIT, ROLLBACK, SAVEPOINT

1. SELECT — Get Data

SELECT is probably the SQL command you'll use most.

Basic

```
SELECT *  
FROM employee;
```

Meaning:

Give all columns and all employees.

Select specific columns

```
SELECT employee_id, employee_name, salary  
FROM employee;
```

2. INSERT — Add Data

INSERT adds a new record.

```
INSERT INTO employee  
(employee_id, employee_name, email, salary, hire_date, department_id)  
VALUES  
(101, 'Rohit', 'rohit@example.com', 50000, '2026-09-11', 1);
```

3. UPDATE — Change Existing Data

```
UPDATE employee  
SET salary = 60000  
WHERE employee_id = 101;
```

Only employee 101 is updated the employee here is table and salary is col and emp_id is also a column.

Very important developer rule

Never casually write:

```
UPDATE employee  
SET salary = 60000;
```

This changes every employee's salary.

4. DELETE — Remove Data

```
DELETE FROM employee  
WHERE employee_id = 101;
```

This deletes employee 101.

Developer thinking

Imagine:

```
DELETE /employees/101
```

The backend might execute:

```
DELETE FROM employee  
WHERE employee_id = 101;
```

Dangerous

```
DELETE FROM employee;
```

This deletes **all rows**.

5. WHERE — Filter Data

WHERE tells SQL:

SQL provides the WHERE clause to filter rows based on one or more conditions. It ensures that queries return or modify only the required records.

Salary greater than 50,000

```
SELECT *  
FROM employee  
WHERE salary > 50000;
```

Specific department

```
SELECT *  
FROM employee  
WHERE department_id = 2;
```

Multiple conditions

```
SELECT *  
FROM employee  
WHERE salary > 50000  
AND department_id = 2;
```

OR

```
SELECT *  
FROM employee  
WHERE department_id = 1  
OR department_id = 2;
```

6. ORDER BY — Sort Data

Suppose we want employees from highest salary to lowest.

```
SELECT employee_name, salary  
FROM employee  
ORDER BY salary DESC;
```

DESC = descending.

ASC = ascending.

We can use DESC and ASC according to our needs.

Multiple sorting

```
SELECT employee_name, salary, hire_date  
FROM employee  
ORDER BY salary DESC, hire_date ASC;
```

GROUP BY

Definition:

GROUP BY is used to **group rows having the same value** and perform aggregate calculations on each group.

Used with: COUNT(), SUM(), AVG(), MIN(), MAX()

Example:

```
SELECT department_id, COUNT(*) AS total_employees
FROM employee
GROUP BY department_id;
```

Note:

- **GROUP BY** creates groups based on a column.
- Aggregate functions calculate a value for each group.
- Usually used with **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**.
- Use **HAVING** to filter the groups.

JOIN**Definition:**

JOIN is used to combine data from two or more tables based on a related column.

Example:

```
SELECT e.employee_name, d.department_name
FROM employee e
JOIN department d
ON e.department_id = d.department_id;
```

Note:

- **JOIN** connects related tables.
- **ON** specifies the matching condition.
- Common types: **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN**.
- Useful when related data is stored in different tables.

JOIN — Types**1. INNER JOIN**

Returns only the records that match in both tables.

```
SELECT *
FROM employee e
INNER JOIN department d
ON e.department_id = d.department_id;
```

Note: Only matching records are returned.

2. LEFT JOIN

Returns **all records from the left table** and matching records from the right table.

```
SELECT *
FROM employee e
LEFT JOIN department d
ON e.department_id = d.department_id;
```

Note: If there is no match, right-table columns become NULL.

3. RIGHT JOIN

Returns **all records from the right table** and matching records from the left table.

```
SELECT *  
FROM employee e  
RIGHT JOIN department d  
ON e.department_id = d.department_id;
```

Note: If there is no match, left-table columns become NULL.

4. FULL OUTER JOIN

Returns **all records from both tables**, whether they match or not.

```
SELECT *  
FROM employee e  
FULL OUTER JOIN department d  
ON e.department_id = d.department_id;
```

Note: Unmatched columns contain NULL.

5. CROSS JOIN

Returns **every possible combination** of rows from both tables.

```
SELECT *  
FROM employee  
CROSS JOIN department;
```

Note: If Employee has 5 rows and Department has 3 rows → $5 \times 3 = 15$ rows.

6. SELF JOIN

A table is joined **with itself**.

Example: finding an employee's manager when both are stored in the employee table.

```
SELECT  
    e.employee_name AS employee,  
    m.employee_name AS manager  
FROM employee e  
JOIN employee m  
ON e.manager_id = m.employee_id;
```

Aggregate Functions in SQL

Aggregate functions are used when you want to **calculate something from multiple rows and return one result**.

```
COUNT()  
SUM()  
AVG()  
MIN()  
MAX()
```

like:

Many rows

↓

Aggregate Function

↓

One calculated result

Example Employee Table

Let's use this throughout:

id	name	department	salary
1	Rohit	IT	50000
2	Aman	IT	60000
3	Priya	HR	45000
4	Rahul	HR	55000
5	Neha	Sales	40000

1. COUNT()

Purpose

COUNT () tells you how many rows/values exist.

Count all employees

```
SELECT COUNT(*)  
FROM employee;
```

Result:

5

COUNT with GROUP BY

Business question:

How many employees are in each department?

```
SELECT  
    department,  
    COUNT(*) AS employee_count  
FROM employee  
GROUP BY department;
```

Result:

department	employee_count
IT	2
HR	2

department	employee_count
Sales	1

This is a very common real-world query.

2. SUM()

Purpose

SUM() calculates the **total** of a numeric column.

For example, total salary:

```
SELECT SUM(salary) AS total_salary
FROM employee;
```

Result:

250000

SUM with GROUP BY

Business question:

What is the total salary expense for each department?

```
SELECT
    department,
    SUM(salary) AS total_salary
FROM employee
GROUP BY department;
```

Result:

department	total_salary
IT	110000
HR	100000
Sales	40000

3. AVG()

Purpose

AVG() calculates the **average value**.

```
SELECT AVG(salary) AS average_salary
FROM employee;
```

Calculation:

250000 / 5 = 50000

Result:

50000

AVG by department

```
SELECT
    department,
    AVG(salary) AS average_salary
FROM employee
GROUP BY department;
```

Result:

department	average_salary
IT	55000
HR	50000
Sales	40000

business use

"Which department has the highest average salary?"

```
SELECT
    department,
    AVG(salary) AS average_salary
FROM employee
GROUP BY department
ORDER BY average_salary DESC;
```

4. MIN()

Purpose

MIN() finds the smallest value.

```
SELECT MIN(salary) AS lowest_salary
FROM employee;
```

Result:

40000

MIN by department

```
SELECT
    department,
    MIN(salary) AS lowest_salary
FROM employee
GROUP BY department;
```

Result:

department	lowest_salary
IT	50000

department	lowest_salary
HR	45000
Sales	40000

5. MAX()

Purpose

MAX() finds the **largest value**.

```
SELECT MAX(salary) AS highest_salary
FROM employee;
```

Result:

60000

MAX by department

```
SELECT
    department,
    MAX(salary) AS highest_salary
FROM employee
GROUP BY department;
```

Result:

department	highest_salary
IT	60000
HR	55000
Sales	40000

COUNT → How many?

SUM → How much total?

AVG → What's the average?

MIN → What's the lowest?

MAX → What's the highest?

The Real Power: Combining Them

As a developer, you will often use several aggregate functions together.

Business question:

Give me the number of employees, total salary, average salary, lowest salary, and highest salary.

```
SELECT
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MIN(salary) AS lowest_salary,
    MAX(salary) AS highest_salary
FROM employee;
```

Result:

employee_count	total_salary	average_salary	lowest_salary	highest_salary
5	250000	50000	40000	60000

That's a **summary/analytics query**.

Aggregate Functions + GROUP BY

```
SELECT
    department,
    COUNT(*) AS employees,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM employee
GROUP BY department;
```

Result:

department	employees	total_salary	average_salary	minimum_salary	maximum_salary
IT	2	110000	55000	50000	60000
HR	2	100000	50000	45000	55000
Sales	1	40000	40000	40000	40000

Now you've created a **department-level business report directly from SQL**.

Aggregate Functions + HAVING

Business question:

Find departments with more than 1 employee.

```
SELECT
    department,
    COUNT(*) AS employee_count
FROM employee
GROUP BY department
HAVING COUNT(*) > 1;
```

Short explanation of 5 complex queries

1. Q12 — Subquery + AVG

```
-- Q12
-- Which employees have a salary above the company average?
SELECT employee_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary)
    FROM employee
);
```

Explanation:

The inner query calculates the average salary of all employees. The outer query then finds employees whose salary is greater than that average. This helps the company identify employees earning above the overall salary level.

2. Q18 — GROUP BY + COUNT + HAVING

```
-- Q18
-- Which departments have more than 1 employee?
SELECT department_id, COUNT(*) AS employee_count
FROM employee
GROUP BY department_id
HAVING COUNT(*) > 1;
```

Explanation:

Employees are grouped by department, and COUNT () calculates the number of employees in each department. HAVING then filters the groups and keeps only departments with more than one employee.

3. Q23 — JOIN + WHERE

```
-- Q23
-- Which employees report to Amit Sharma?
SELECT
    e.employee_name,
    m.manager_name
FROM employee e
JOIN manager m
    ON e.manager_id = m.manager_id
WHERE m.manager_name = 'Amit Sharma';
```

Explanation:

The query joins the employee and manager tables using manager_id. The WHERE condition then filters the result to show only employees who report to Amit Sharma.

4. Q28 — JOIN + GROUP BY + SUM + ORDER BY

```
-- Q28
-- Which department has the highest total salary expense?
SELECT
    d.department_name,
    SUM(e.salary) AS total_salary
FROM employee e
JOIN department d
```

```
    ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY total_salary DESC
LIMIT 1;
```

Explanation:

The query joins employees with their departments and calculates the total salary for each department using `SUM( )`. The departments are sorted from highest to lowest salary expense, and `LIMIT 1` returns the department with the highest total.

5. Q30 — JOIN + GROUP BY + COUNT + AVG + HAVING

```
-- Q30
-- Which departments have more than 1 employee and what is their average salary?
SELECT
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    AVG(e.salary) AS average_salary
FROM employee e
JOIN department d
    ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 1;
```

Explanation:

The query joins employees with departments, groups employees by department, counts employees, and calculates their average salary. `HAVING` ensures that only departments with more than one employee are included.